Advanced Data Structures and Algorithm Analysis

Laboratory Project Report · Project 3

Recover the Design

Group X

Student A **Student B**

Zhejiang University

Date: 2026-11-02

Abstract

In this project, we study **fence recovery** on an $n \times m$ grid, where straight horizontal and vertical fences must satisfy the target degree $k \in \{1, 2, 3, 4\}$ of every connector without bending or intersecting at empty cells. We formulate the task as selecting a **degree-constrained subgraph** of **non-crossing edges** on a sparse **candidate graph**: each connector emits at most one rightward edge and one downward edge to its nearest visible neighbors, bounding the candidate set by $|E| \le 2|V|$. We then apply recursive **backtracking** with four **pruning** rules: a root handshaking parity check, a degree upper-bound check, a lookahead lower-bound check $\text{current}(v) + r(v, i) \ge d(v)$ over remaining incident edges, and an open-interval crossing check. We evaluate our solver on the three official samples, a scaling suite `L05`–`L20` and `U05`–`U20` from 5×5 to 20×20, and 360 100-island instances on a 16×16 grid adapted from the **Hashiwokakero** benchmark into solvable single-fence boards. Every output on the solvable suites passes structural verification, and all odd-degree unsolvable boards are rejected in a single call. Across the 360 benchmark instances where $|E| \in [168, 170]$, our solver explores a median of **931 recursive calls** in **0.36** ms of pure search time and 53.90 ms of end-to-end runtime. Finally, our analysis demonstrates the advantages of pruned backtracking: combining inclusion-first branching with lookahead degree bounds cuts infeasible subtrees early and keeps the actual search tree close to linear in $|E|$ despite the exponential worst case.

Keywords: Fence recovery; degree-constrained subgraph; non-crossing edges; candidate graph; backtracking; pruning; Hashiwokakero.

Contents

1 Introduction **1**
1.1 Problem Description . . . 1
1.2 Purpose of This Report . . . 1

2 Data Structure and Algorithm Specification **2**
2.1 Overview of Board and Graph Representations . . . 2
2.2 Candidate Graph Construction . . . 2
2.3 Backtracking Search and Pruning Rules . . . 3
2.4 Directional Output Formatting . . . 4

3 Testing Results **5**
3.1 Dataset Links and Benchmark Specifications . . . 5
3.2 Test Suite and Structural Verification . . . 5
3.3 Performance Benchmark Setup and Runtime Tables . . . 6
3.4 Runtime Plots and Recovered Fence Layouts . . . 6

4 Analysis and Comments **8**
4.1 Time and Space Complexity Analysis . . . 8
4.2 Why Pruned Backtracking Performs Well on Benchmark Instances . . . 8
4.3 Further Possible Improvements . . . 9

Author List and Declaration **10**

References **10**

A Source Code **11**
A.1 Graph and Pruning (`recover_the_design.h`) . . . 11
A.2 Search Driver (`recover_the_design.c`) . . . 14
A.3 Windows Batch Timing Driver (`testbench_time.c`) . . . 15
A.4 Pure Backtracking Profiler (`testbench_profile.c`) . . . 18
A.5 Structural Output Verifier (`verify_output.py`) . . . 20
A.6 Benchmark and Layout Visualization Script (`visualization.py`) . . . 21

Chapter 1. Introduction

1.1 Problem Description

In an $n \times m$ garden with $1 \le n, m \le 50$, some cells contain connectors of degree 1 to 4 and the rest are empty. The original fence drawing is lost; the task is to reconstruct straight horizontal and vertical fences that meet every connector's degree without bending or crossing in an empty cell, as illustrated in Figure 1. Given the degree matrix, the program outputs the four connection directions `up`, `down`, `left`, and `right` of each connector in row-major order, or `No Solution` if no valid layout exists.

This layout is closely related to the puzzle Hashiwokakero (橋を架けろ), where islands of given degrees are joined by non-crossing straight bridges [1]. Unlike the classical puzzle, each direction here carries at most one fence and the fences need not form a single connected network. To solve this problem, we apply the backtracking method from the ADS course [3]:

- **Backtracking:** Since every fence must end at the nearest visible connector in its row or column, all possible fences form a sparse candidate graph in which each edge is either selected or skipped. Depth-first search decides the candidate edges one by one and prunes a branch as soon as a connector exceeds its degree, lacks enough remaining edges to reach its degree, or introduces a crossing [3, 4].

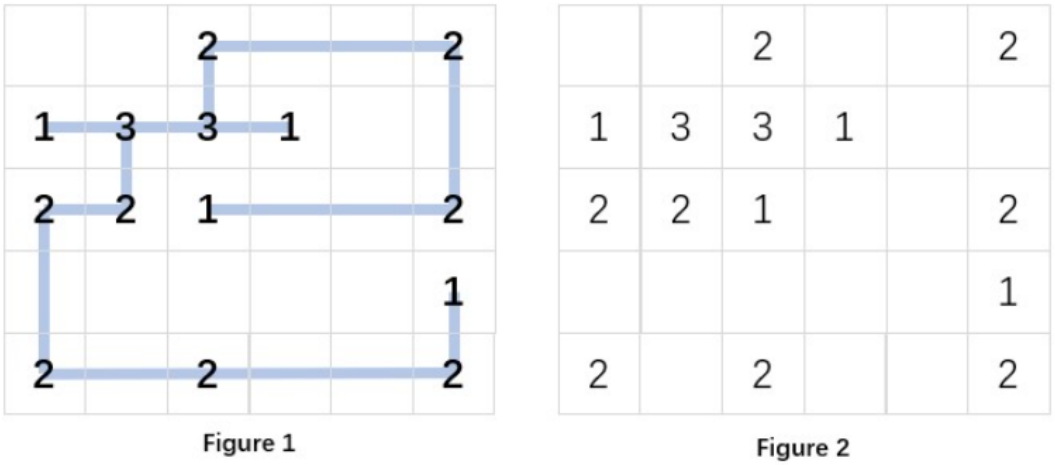


Figure 1: The assignment example.

1.2 Purpose of This Report

The goal of this project is to recover one feasible drawing by pruned backtracking and to measure that search on the boards we were given. Specifically, we:

1. **Build the Candidate Graph and the Search:** Implement nearest-visible candidate fences and the pruned depth-first search in `recover_the_design.h` and `recover_the_design.c`.
2. **Check Three Families of Boards:** Run the three official samples, the constructed boards `L05`–`L20` and `U05`–`U20`, and the 360 revised 100-island instances from the Coelho–Laporte–Lindbeck–Vidal benchmark [1].
3. **Profile Scaling and Pruning Efficiency:** Measure recursive call counts and execution times across grid scales and analyze how lookahead pruning and degree slack govern search performance.

Chapter 2. Data Structure and Algorithm Specification

2.1 Overview of Board and Graph Representations

Table 1 summarizes the core C data structures defined in `src/recover_the_design.h`. All grid cells are stored in a contiguous row-major array indexed by `FlattenIndex`$(x, y, m) = x \cdot m + y$. Empty cells remain in `vertices[]` with `degree` $= 0$, allowing row and column scans to check cell occupancy in $O(1)$ time without maintaining a separate grid matrix.

Table 1: Summary of board and candidate-graph data structures implemented in `src/recover_the_design.h`.

Structure	C Fields / Layout	Structural Invariant & Role
`Vertex`	`int degree`, `int x`, `int y`	Represents one grid cell at 0-based coordinates (x, y); `degree` $\in \{1, 2, 3, 4\}$ denotes a connector and `degree` $= 0$ denotes an empty cell.
`Edge`	`Vertex from`, `Vertex to`, `int type`, `bool is_selected`	Undirected candidate fence between two nearest visible connectors; `type` is `horizontal` or `vertical`, and `is_selected` tracks whether the fence is active on the current search path.
`Graph`	`rows`, `columns`, `edge_num`, `vertex_num`, `int* current_degree`, `Vertex* vertices`, `Edge* edges`	Holds the flattened cell array `vertices[]`, the candidate array `edges[]`, and the active degree counter `current_degree[]` during backtracking.
`result`	`int x`, `int y`, `bool left`, `right`, `up`, `down`	Stores the four directional connection flags of each cell for row-major output formatting in `show_result`.

2.2 Candidate Graph Construction

Because a valid fence cannot bend or pass through an intermediate connector without terminating there, each connector only needs to be linked to its nearest visible connector in each of the four cardinal directions. Moreover, an undirected horizontal fence between (x, y_1) and (x, y_2) with $y_1 < y_2$ is shared by both endpoints, and the same holds for a vertical fence between (x_1, y) and (x_2, y) with $x_1 < x_2$. Consequently, `init_Graph` scans the board in row-major order and emits at most two candidate edges from each connector: one to the nearest visible connector on its right, and one to the nearest visible connector below it.

Algorithm 1: Candidate Graph Construction init_Graph(G, n, m) in `recover_the_design.h`

1. Allocate `vertices`$[0..nm-1]$, `current_degree`$[0..nm-1] \leftarrow 0$, and `edges`$[0..4nm-1]$; read the $n \times m$ degree matrix into `vertices[]`.
2. **for each** cell $u = (x, y)$ in row-major order with u.degree > 0 **do**
3. Advance $y' \leftarrow y + 1$ while $y' < m$ and `vertices`$[x \cdot m + y']$.degree $= 0$.
4. **if** $y' < m$ **then** append horizontal candidate edge $\big(u, (x, y')\big)$ to `edges[]` and increment `edge_num`.
5. Advance $x' \leftarrow x + 1$ while $x' < n$ and `vertices`$[x' \cdot m + y]$.degree $= 0$.
6. **if** $x' < n$ **then** append vertical candidate edge $\big(u, (x', y)\big)$ to `edges[]` and increment `edge_num`.

Since every connector contributes at most one rightward edge and one downward edge, the total number of candidate edges satisfies

$$|E| \le 2|V| \le 2nm.$$

On the 100-island benchmark boards, this bound is tight: every instance produces between 168 and 170 candidate edges.

2.3 Backtracking Search and Pruning Rules

Given the candidate array `edges`$[0..|E|-1]$, `backtrack` explores binary decisions from index 0 to $|E|-1$, trying the inclusion branch `is_selected` = `true` before the exclusion branch `is_selected` = `false`. Let current(v) be the number of selected fences currently incident to v, and let $r(v,i)$ be the number of undecided candidate edges in `edges`$[i+1..|E|-1]$ incident to v, computed by `count_remaining_edges`. Four pruning rules cut infeasible branches before reaching a leaf:

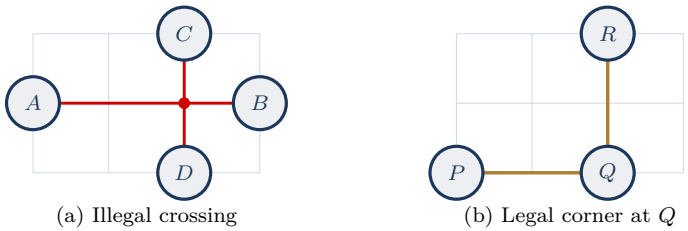
- **Handshaking Parity Check:** Every selected fence adds 2 to the total degree across all connectors. At the root call $i=0$, if $\sum_v d(v)$ is odd, `backtrack` immediately returns `false` [2].
- **Degree Upper-Bound Check:** When edge $i=(u,w)$ is tentatively selected, `check_edge` rejects the branch if current$(u) > d(u)$ or current$(w) > d(w)$.
- **Lookahead Lower-Bound Check:** A connector v can still reach its target degree $d(v)$ only if

$$\text{current}(v) + r(v,i) \;\geq\; d(v).$$

  This bound is enforced both inside `check_edge` on the inclusion branch and in `backtrack` before entering the exclusion branch.
- **Orthogonal Crossing Check:** Two parallel edges never cross. A horizontal edge on row x_h spanning columns $[y_1, y_2]$ and a vertical edge on column y_v spanning rows $[x_1, x_2]$ cross at an empty interior cell if and only if

$$y_1 < y_v < y_2 \quad \text{and} \quad x_1 < x_h < x_2.$$

  As shown in Figure 2, sharing a connector endpoint corresponds to equality at a boundary and is not treated as a crossing.



(a) Illegal crossing (b) Legal corner at Q

Figure 2: Crossing check.

Algorithm 2: Recursive Search Backtrack(G,i) and check_edge(G,i)

Backtrack:

1. **if** $i=0$ and $\sum_v d(v)$ is odd **then return** `false`.
2. **if** $i=|E|$ **then return** `true` if current$(v)=d(v)$ for all v, else `false`.
3. Set `edges`$[i]$.is_selected $\leftarrow$ `true` and increment current(u), current(w) for endpoints u, w.
4. **if** check_edge(G,i) and Backtrack$(G,i+1)$ **then return** `true`.
5. Set `edges`$[i]$.is_selected $\leftarrow$ `false` and decrement current(u), current(w).
6. **if** current$(u)+r(u,i) \geq d(u)$ and current$(w)+r(w,i) \geq d(w)$ **and** Backtrack$(G,i+1)$ **then return** `true`.
7. **return** `false`.

Check-Edge:

1. **if** current$(u) > d(u)$ or current$(w) > d(w)$ **then return** `false`.
2. **if** current$(u)+r(u,i) < d(u)$ or current$(w)+r(w,i) < d(w)$ **then return** `false`.
3. **for** $j=0$ **to** $|E|-1$ **do if** `edges`$[j]$.is_selected and is_crossing(`edges`$[j]$, `edges`$[i]$) **then return** `false`.
4. **return** `true`.

2.4 Directional Output Formatting

Once `backtrack` returns `true`, `show_result` allocates a zero-initialized array `res`$[0..nm-1]$ of `result` records. Each selected horizontal edge sets `right` = `true` on its left endpoint and `left` = `true` on its right endpoint; each selected vertical edge sets `down` = `true` on its upper endpoint and `up` = `true` on its lower endpoint. Scanning the grid in row-major order and skipping empty cells prints each connector in the required 1-based format `row col up down left right`.

Chapter 3. Testing Results

3.1 Dataset Links and Benchmark Specifications

We evaluate our solver on three groups of puzzle boards:

- **Official Problem Samples:** Three small test cases from the problem specification: a solvable 5×6 board shown in Figure 1, an unsolvable 5×6 board with an odd degree sum, and a 3×5 board that admits two distinct valid drawings.
- **Constructed Scaling Suite, `recover_the_design_large_tests.txt`:** Sixteen solvable $n \times n$ boards `L05`–`L20` with $n \in [5, 20]$, together with four unsolvable odd-degree boards `U05`, `U10`, `U15`, and `U20`.
- **Revised 100-Island Hashiwokakero Benchmark:** Obtained from the public benchmark of Coelho et al. [1] at `https://w1.cirrelt.ca/~vidalt/en/research-data.html`, which provides 360 100-island instances on a 16×16 grid. Because the original puzzles allow double bridges and island degrees up to 8, we modified them into solvable single-fence instances under `data/Hashi_Puzzles/100_revised/` by keeping all 100 island positions fixed, removing the island-count header token, and adjusting positive degrees into $\{1, 2, 3, 4\}$ from a valid non-crossing single-fence layout stored in `100_solutions/`.

3.2 Test Suite and Structural Verification

Because a degree matrix can admit multiple valid fence layouts, our checker `verify_output.py` validates the output against the four layout rules rather than requiring byte-identical text: every connector must appear once in row-major order, the four directional flags at each connector must sum to its required degree, every active direction must match the opposite flag on the nearest visible connector across empty cells, and no horizontal fence may cross a vertical fence. Table 2 summarizes all test cases.

Table 2: Summary of functional, structural, and large-scale performance test cases.

ID	Test Purpose	Expected Result	Actual Behavior	Status
TC-1	Official 5×6 Sample 1 in Figure 1	Valid 14-line connector layout matching Sample Output 1	Output matches Sample Output 1 byte for byte; 13 fences, 19 calls	Pass
TC-2	Official 5×6 Sample 2 with cell $(3, 6)$ lowered to 1	`No Solution`	Odd degree sum 25 rejected at root in 1 call	Pass
TC-3	Official 3×5 Sample 3 with multiple valid layouts	Either of the two valid 8-fence drawings	Returns Sample Output 3 in 16 calls; verified by `verify_output.py`	Pass
TC-4	Solvable scaling suite `L05`–`L20`, 5×5 to 20×20	All degree, visibility, and non-crossing rules hold	All 16 boards pass verification; $\|V\|$ grows from 21 to 280 and $\|E\|$ from 32 to 520	Pass
TC-5	Unsolvable boards `U05`, `U10`, `U15`, and `U20`	`No Solution`	Odd degree sums 51, 171, 379, and 755 each rejected in 1 call	Pass
TC-6	Structural verification on 360 revised 100-island boards	Valid layout for every `.has` instance	All 360 outputs pass `verify_output.py`; median 931 calls	Pass
TC-7	Batch runtime logging via `testbench_time.c`	Complete CSV log without timeout	All 360 runtimes logged in `benchmark_results_100.csv`; median 53.90 ms, max 189.88 ms	Pass

In TC-2, lowering the connector at $(3, 6)$ from 2 to 1 changes the total degree from 26 to 25, so the root parity check in `backtrack` rejects the board before testing any candidate edge. In TC-3, the 3×5 board

has exactly two valid drawings of eight fences, which differ in whether the degree-1 connector at $(1, 1)$ connects rightward to $(1, 2)$ or downward to $(2, 1)$. Because `init_Graph` emits the rightward edge first and `backtrack` tries inclusion before exclusion, our solver outputs the rightward layout matching Sample Output 3.

3.3 Performance Benchmark Setup and Runtime Tables

To evaluate both algorithmic search cost and end-to-end execution time, we record two sets of measurements:

- **Pure Backtracking Search Profile:** Compiled with `gcc -O2`, `testbench_profile.c` instruments `backtrack` to record the number of candidate edges $|E|$, selected fences $|E^*|$, recursive calls, and in-memory search time excluding process creation and file I/O.
- **End-to-End Batch Timing:** On Windows, `testbench_time.c` spawns `recover_the_design.exe` once per `.has` file with a 60 s timeout and measures wall-clock time via `QueryPerformanceCounter`, writing all 360 records to `benchmark_results_100.csv`.

Table 3: Backtracking search profile on the constructed boards `L05`–`L20`.

Board	Grid	Connectors $\|V\|$	Candidates $\|E\|$	Selected $\|E^*\|$	Calls	Search Time, ms
L05	5×5	21	32	25	37	0.015
L08	8×8	38	60	43	66	0.020
L10	10×10	67	114	85	197	0.153
L12	12×12	101	178	126	536	0.254
L14	14×14	140	252	187	430	0.244
L16	16×16	177	322	239	754	0.536
L18	18×18	229	422	304	8,544	8.340
L20	20×20	280	520	377	1,238	1.360

Table 4: Distribution summary of structural and runtime metrics across the 360 revised 100-island benchmark instances on a 16×16 grid.

Metric	Min	25th Pct	Median	Mean	SD	95th Pct	Max
Candidate Edges $\|E\|$	168.00	168.00	168.00	168.23	0.48	169.00	170.00
Selected Fences $\|E^*\|$	75.00	87.00	91.00	91.23	5.83	101.00	105.00
Recursive Calls T	172.00	375.75	930.50	3,455.28	9,580.19	13,312.30	110,997.00
Pure Search Time, ms	0.08	0.16	0.36	1.22	3.17	4.44	30.51
End-to-End Runtime, ms	41.71	49.45	53.90	59.40	17.73	91.63	189.88

3.4 Runtime Plots and Recovered Fence Layouts

Figures 3 to 5 present the benchmark plots and recovered fence layouts generated by `visualization.py`. On the constructed boards `L05`–`L20` in Figure 3, $|V|$, $|E|$, and $|E^*|$ grow smoothly with grid width n, and most boards finish within 200–1,400 recursive calls and under 1.4 ms, except for `L18` which explores 8,544 calls in 8.34 ms before finding a valid layout.

Across the 360 revised 100-island instances in Figure 4, every board has 168–170 candidate edges; pure search time has a median of 0.36 ms, a mean of 1.22 ms, and a maximum of 30.51 ms at 110,997 calls, whereas end-to-end process runtime has a minimum of 41.71 ms, a median of 53.90 ms, a mean of 59.40 ms, a 95th percentile of 91.63 ms, and a maximum of 189.88 ms. Finally, Figure 5 visualizes the non-crossing fence layouts recovered by our solver on three representative boards: `L08` (8×8), `L12` (12×12), and Benchmark Instance #1 (`Hs_16_100_25_00_001.has`, 16×16), where solid amber segments denote selected fences and dashed gray segments denote unselected candidate edges.

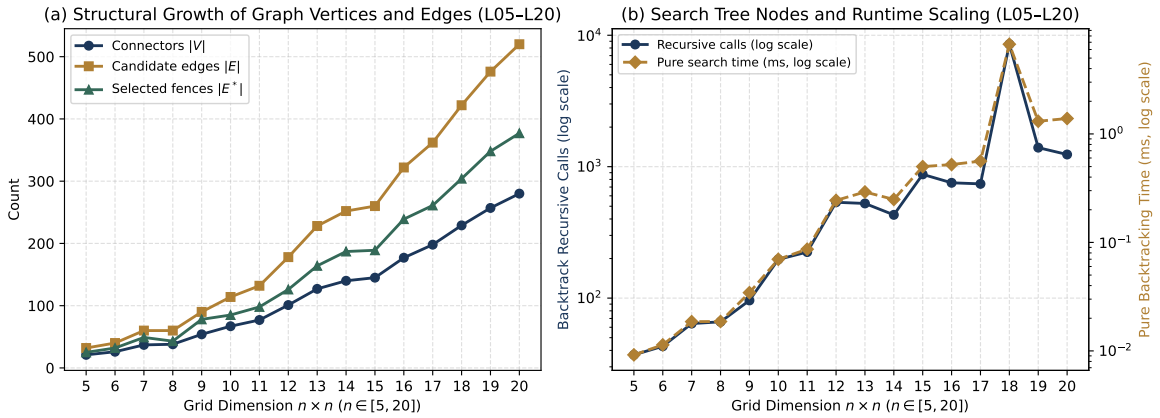


Figure 3: Structural growth and search scaling on L05–L20.

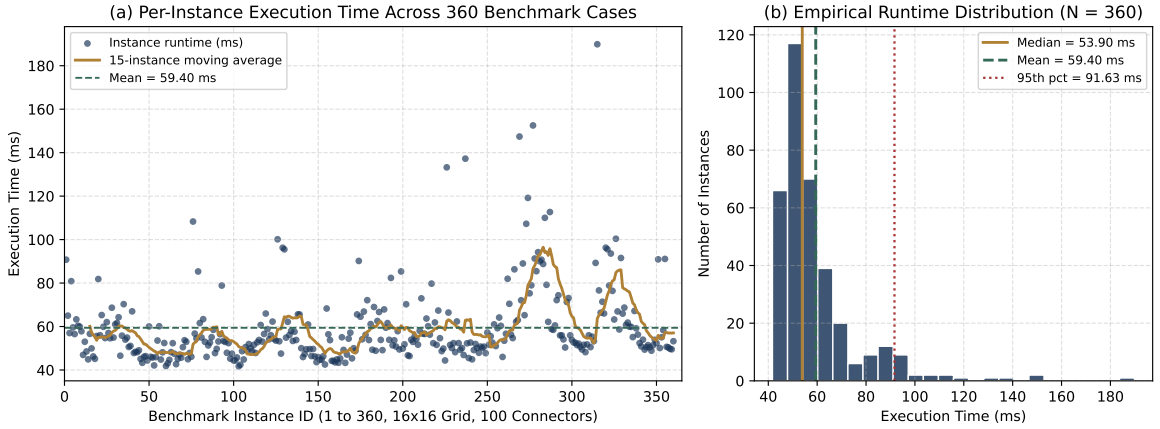


Figure 4: End-to-end execution time across the 360 revised 100-island benchmark instances.

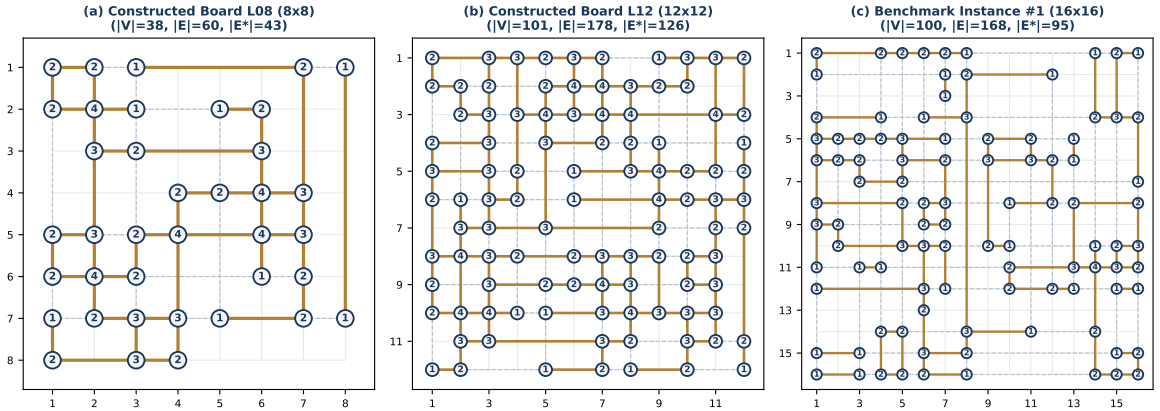


Figure 5: Recovered fence layouts on L08, L12, and Benchmark Instance #1.

Chapter 4. Analysis and Comments

4.1 Time and Space Complexity Analysis

Let T be the number of recursive calls to `backtrack` before the search terminates. Table 5 summarizes the theoretical time and space complexities of each stage of our solver on an $n \times m$ board with $|V|$ connectors and $|E|$ candidate fences.

Table 5: Theoretical time and space complexities of candidate-graph construction and pruned backtracking.

Stage / Operation	Count per Instance	Time Complexity	Auxiliary Space
Grid & Candidate Init (`init_Graph`)	1	$\Theta(nm)$	$\Theta(nm)$
Root Parity Check	1	$\Theta(nm)$	$\Theta(1)$
Lookahead & Crossing Check (`check_edge`)	$\leq T$	$O(\|E\|)$	$\Theta(1)$
Leaf Verification ($i = \|E\|$)	$\leq T$	$\Theta(nm)$	$\Theta(1)$
Output Formatting (`show_result`)	1	$\Theta(nm + \|E\|)$	$\Theta(nm)$
Unpruned Search Tree	$2^{\|E\|+1} - 1$ nodes	$O(\|E\| \cdot 2^{\|E\|})$	$\Theta(nm + \|E\|)$
Pruned Backtracking Search	$T \leq 2^{\|E\|+1} - 1$	$O(\|E\| \cdot T + nm)$	$\Theta(nm + \|E\|)$

Time Complexity Details. In `init_Graph`, reading the $n \times m$ grid takes $\Theta(nm)$ time, and scanning rightward and downward from each connector visits only empty cells before stopping at the first visible neighbor, so each empty cell is traversed at most once horizontally and once vertically, totaling $\Theta(nm)$ time. Inside each recursive call at index i, `count_remaining_edges` scans the suffix `edges`$[i+1..|E|-1]$ for both endpoints in $O(|E|)$ time, and `check_edge` scans `edges`$[0..|E|-1]$ to test crossings in $O(|E|)$ time. Because $|E| \leq 2|V| \leq 2nm$, the worst-case unpruned search takes $O(|E| \cdot 2^{|E|})$ time, whereas our pruned search takes $O(|E| \cdot T + nm)$ time.

Space Complexity Details. `init_Graph` allocates `vertices`$[0..nm-1]$, `current_degree`$[0..nm-1]$, and `edges`$[0..4nm-1]$, while `show_result` allocates `res`$[0..nm-1]$; all four arrays take $\Theta(nm)$ space. During recursion, the call stack reaches depth at most $|E| + 1 \leq 2|V| + 1$ with $O(1)$ local variables per frame, so total auxiliary space is $\Theta(nm + |E|) = \Theta(nm)$.

4.2 Why Pruned Backtracking Performs Well on Benchmark Instances

Two empirical findings from Chapter 3 explain both the speed of the recursion and the variance across individual boards.

Key Finding 1: Inclusion-First Ordering and Lookahead Pruning Keep T Near $|E|$

Trying the inclusion branch before the exclusion branch constructs a valid layout from the earliest feasible candidates, while the lookahead bound $\text{current}(v) + r(v, i) \geq d(v)$ immediately cuts the exclusion branch whenever skipping an edge leaves either endpoint short of its required degree. Consequently, `L05` explores only 37 calls over 32 edges, `L20` resolves 520 edges in 1,238 calls, and the median 100-island board visits 930.5 nodes over 168 edges, fewer than 6 calls per candidate edge.

Key Finding 2: Search Cost Tracks Degree Slack Rather Than Grid Size

Across all 360 revised 100-island boards, the candidate-edge count $|E|$ stays nearly constant at 168–170, yet the number of selected fences $|E^*|$ ranges from 75 to 105. Boards with fewer required fences have larger degree slack between the number of incident candidates and $d(v)$, allowing both the inclusion and exclusion branches to pass the lookahead test deeper into the tree and producing the right-skewed tail up to 110,997 calls. For the same reason, `L18` requires 8,544 calls whereas the larger board `L20` finishes in 1,238 calls.

4.3 Further Possible Improvements

Based on our profiling of `src/`, two improvements can further accelerate the solver:

1. **Incremental Remaining-Edge Counters and Grid Occupancy Map:** Maintaining a remaining-edge counter per vertex updated in $O(1)$ time at each step, together with a 2D occupancy array for active fences, would replace the $O(|E|)$ suffix and crossing scans inside `check_edge`.
2. **Most-Constrained Edge Ordering:** Dynamically branching on candidate edges incident to connectors whose remaining degree slack $r(v, i) - (d(v) - \text{current}(v))$ is smallest would trigger lookahead pruning earlier on hard instances such as `L18` and the long-tail benchmark boards [4].

Author List and Declaration

Author List

- **Student A:** Implemented the candidate-graph construction, the pruned backtracking search, and the timing driver.
- **Student B:** Checked the recovered drawings, profiled the search on the benchmark instances, and wrote this report.

Declaration

We hereby declare that all the work done in this project titled "Recover the Design" is of our independent effort as a group.

Signatures

Student A Student B

References

[1] Leandro C. Coelho et al. "Benchmark Instances and Branch-and-Cut Algorithm for the Hashiwokakero Puzzle". In: *Technical Report CIRRELT-2019-16 / arXiv:1905.00973* (2019).

[2] Thomas H. Cormen et al. *Introduction to Algorithms*. 4th ed. MIT Press, 2022.

[3] Course Teaching Team of Advanced Data Structures and Algorithm Analysis. *Course Lecture Slides: Backtracking, Branch-and-Bound, and NP-Completeness*. College of Computer Science and Technology, Zhejiang University. 2026.

[4] Mark Allen Weiss. *Data Structures and Algorithm Analysis in C*. 2nd ed. Addison-Wesley, 1996.

Appendix A. Source Code

All complete source files are located in `project3/src/`. Below we include the candidate-graph and pruning header, the backtracking search driver, the Windows batch timing driver, the cross-platform pure-search profiler, the independent structural output verifier, and the Python plotting script.

A.1 Graph and Pruning (`recover_the_design.h`)

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

/*
 * Recover the Design.
 * The board is stored as a flat array of cells. init_Graph links every
 * connector to its nearest visible neighbor on the right and below, so
 * each possible fence is one undirected edge. backtrack, in the .c file,
 * then chooses a subset of those edges. A subset is kept only when every
 * connector has its required degree and no two chosen fences cross in an
 * empty cell. The search tries taking an edge before skipping it, and it
 * drops a branch as soon as a degree can no longer be met.
 */
#define horizontal 0
#define vertical 1

typedef struct{
    int degree;
    int x;
    int y;
}Vertex; /* degree is the required number of fences; (x, y) is 0-based. */


typedef struct{
    Vertex from;
    Vertex to;
    int type;
    bool is_selected; /* True when this fence is taken on the current path. */
}Edge; /* from is the left or upper end; type is horizontal or vertical. */


typedef struct{
    int rows;
    int columns;
    int edge_num;
    int vertex_num;
    int *current_degree; /* Fences taken so far at each flattened cell. */
    Vertex* vertices;
    Edge* edges;
}Graph; /* vertices[] covers every cell; edges[] holds only candidates. */


typedef struct{
    int x;
    int y;
    bool left;
    bool right;
    bool up;
    bool down;
}result; /* One output record: row, column, and the four direction flags. */



int FlattenIndex(int x, int y, int columns) {
    return x * columns + y;
} /* Row-major index x * columns + y. */


/*
 * Read the degree matrix and build the candidate list.
 * A connector emits at most one edge to the right and one edge downward,
```

```
 * each ending at the nearest connector in that direction.
 */
void init_Graph(Graph* g, int rows,int columns){
    g->rows = rows;
    g->columns = columns;
    g->edge_num = 0;
    g->vertex_num = 0;
    g->current_degree = (int*)malloc(rows * columns * sizeof(int)); /* One counter per cell. */
    for(int i = 0; i < rows * columns; i++){
        g->current_degree[i] = 0;
    }
    g->vertices = (Vertex*)malloc(rows * columns * sizeof(Vertex)); /* One slot per cell, empty or not. */
    g->edges = (Edge*)malloc(rows * columns *4 * sizeof(Edge)); /* Upper bound: four stubs per cell. */
    /* Read the n by m degree matrix. A positive entry is a connector. */
    for(int i = 0 ; i < g->rows ; i++){
        for(int j = 0 ; j < g->columns ; j++){
            int degree;
            scanf("%d",&degree);
            if(degree > 0){
                g->vertex_num++;
                Vertex v = {degree, i, j};
                g->vertices[FlattenIndex(i, j, g->columns)] = v; /* Store the connector in row-major order. */
            }
            else{
                Vertex v = {0, i, j};
                g->vertices[FlattenIndex(i, j, g->columns)] = v;
            }
        }
    }
    /* Second pass: emit the rightward edge, then the downward edge. */
    /* Empty cells emit nothing. Each candidate is stored once. */
    for(int k = 0;k < rows*columns;k++){
        Vertex now_vertex = g->vertices[k];
        if(now_vertex.degree > 0){
            int x = now_vertex.x;
            int y = now_vertex.y;
            /* Horizontal candidate: nearest connector strictly to the right. */
            int right_y = y + 1;
            while(right_y < g->columns && g->vertices[FlattenIndex(x, right_y, g->columns)].degree == 0){
                right_y++;
            }
            if(right_y < g->columns){ /* A connector was found before the row ended. */
                Vertex right_vertex = g->vertices[FlattenIndex(x, right_y, g->columns)];
                Edge e = {now_vertex, right_vertex, horizontal,false};
                g->edges[g->edge_num] = e; /* Append; from is the left endpoint. */
                g->edge_num++;
            }
            /* Vertical candidate: nearest connector strictly below. */
            int down_x = x + 1;
            while(down_x < g->rows && g->vertices[FlattenIndex(down_x, y, g->columns)].degree == 0){
                down_x++;
            }
            if(down_x < g->rows){ /* A connector was found before the column ended. */
                Vertex down_vertex = g->vertices[FlattenIndex(down_x, y, g->columns)];
                Edge e = {now_vertex, down_vertex, vertical,false};
                g->edges[g->edge_num] = e; /* Append; from is the upper endpoint. */
                g->edge_num++;
            }
        }
    }
}

/*
 * Two fences cross only when one is horizontal and one is vertical and
 * their open interiors share a cell. Sharing an endpoint is not a cross.
 */
bool is_crossing(Edge a, Edge b) {
    /* Parallel fences share a row or a column, so they do not cross. */
    if (a.type == b.type) {
        return false;
    }

    /* Name the horizontal edge h and the vertical edge v. */
    Edge h = (a.type == horizontal) ? a : b;
    Edge v = (a.type == vertical) ? a : b;
```

```
138
139     /* h runs along row hx from column hy1 to hy2. */
140     /* v runs along column vy from row vx1 to vx2. */
141
142     int hx = h.from.x;
143     int hy1 = h.from.y;
144     int hy2 = h.to.y;
145
146     int vy = v.from.y;
147     int vx1 = v.from.x;
148     int vx2 = v.to.x;
149
150     /* Open interval on both axes: the shared point is an empty cell. */
151     return (hy1 < vy && vy < hy2 &&
152             vx1 < hx && hx < vx2);
153 }
154
155 /*
156  * How many candidate edges after index are incident to v.
157  * Those are the only edges that can still raise v's current degree.
158  */
159 int count_remaining_edges(Graph* g, Vertex v, int index) {
160     int count = 0;
161
162     /* Edges at index or earlier have already been decided. */
163     for (int i = index + 1; i < g->edge_num; i++) {
164         if ((g->edges[i].from.x == v.x &&
165              g->edges[i].from.y == v.y) ||
166             (g->edges[i].to.x == v.x &&
167              g->edges[i].to.y == v.y)) {
168             count++;
169         }
170     }
171
172     return count;
173 }
174
175 /*
176  * Validate taking edges[index]. Three tests, in order:
177  * the edge must not push either endpoint past its required degree,
178  * each endpoint must still be able to reach that degree, and
179  * the edge must not cross a fence already taken.
180  * The skip branch is tested in backtrack, not here.
181  */
182 bool check_edge(Graph* g,int index){
183     if(g->current_degree[FlattenIndex(g->edges[index].from.x, g->edges[index].from.y, g->columns)] > g->edges[index].from.
           degree || g->current_degree[FlattenIndex(g->edges[index].to.x, g->edges[index].to.y, g->columns)] > g->edges[index
           ].to.degree){
184         return false;
185     } /* Taken fences at an endpoint must not exceed its required degree. */
186     if(g->current_degree[FlattenIndex(g->edges[index].from.x, g->edges[index].from.y, g->columns)]+count_remaining_edges(g,
           g->edges[index].from, index) < g->edges[index].from.degree || g->current_degree[FlattenIndex(g->edges[index].to.x,
           g->edges[index].to.y, g->columns)]+count_remaining_edges(g, g->edges[index].to, index) < g->edges[index].to.degree)
           {
187         return false;
188     } /* Current fences plus later candidates must still cover the degree. */
189     /* Compare the new edge with every edge already marked selected. */
190     for(int i = 0;i < g->edge_num;i++){
191         if(g->edges[i].is_selected && is_crossing(g->edges[i],g->edges[index])){
192             return false;
193         } /* A crossing with an already taken fence rejects this edge. */
194     }
195     return true; /* All three tests passed, so the edge may be taken. */
196 }
197
198 /*
199  * Turn taken edges into up/down/left/right flags and print one line per
200  * connector, in row-major order, with coordinates starting at 1.
201  */
202 void show_result(Graph* g){
203     result* res = (result*)calloc(g->rows* g->columns,sizeof(result)); /* Flags start cleared. */
204     int res_index = 0;
205
206     /* Fill coordinates first; direction flags stay false until a fence sets them. */
207     for(int i = 0 ; i < g-> rows;i++){
```

```
        for(int j = 0 ; j < g->columns;j++){
            res_index = FlattenIndex(i, j, g->columns);
            res[res_index].x = i;
            res[res_index].y = j;
        }
    } /* Record the 0-based coordinates of every cell. */
    /* A horizontal fence sets right and left; a vertical fence sets down and up. */
    for(int i = 0;i < g->edge_num;i++){
        if(g->edges[i].is_selected){
            int from_index = FlattenIndex(g->edges[i].from.x, g->edges[i].from.y, g->columns);
            int to_index = FlattenIndex(g->edges[i].to.x, g->edges[i].to.y, g->columns);
            /* Horizontal: left end points right, right end points left. */
            if(g->edges[i].type == horizontal){
                res[from_index].right = true;
                res[to_index].left = true;
            }
            else{ /* Vertical: upper end points down, lower end points up. */
                res[from_index].down = true;
                res[to_index].up = true;
            }
        }
    }

    /* Print connectors only, scanning the board from top to bottom. */
    for(int i = 0 ; i < g-> rows;i++){
        for(int j = 0 ; j < g->columns;j++){
            res_index = FlattenIndex(i, j, g->columns);
            if(g->vertices[res_index].degree == 0){
                continue;
            } /* Empty cells are not printed. */
            /* Coordinates in the output start at 1, not 0. */
            printf("%d %d",res[res_index].x+1,res[res_index].y+1);
            printf(" %d %d %d %d",res[res_index].up,res[res_index].down,res[res_index].left,res[res_index].right);
            printf("\n"); /* row col up down left right, one connector per line. */
        }
    }

}
```

A.2 Search Driver (recover_the_design.c)

```
/*
 * File: recover_the_design.c
 * Description: Depth-first search over the candidate fences built in
 * recover_the_design.h. Each edge is taken or skipped.
 * The first feasible layout is kept. Inclusion is tried
 * before exclusion, which fixes which drawing is printed.
 */

#include "recover_the_design.h"
/*
 * Decide edges[index .. edge_num - 1].
 * The taken branch is tried before the skipped branch.
 * A skipped edge is explored only when both endpoints can still
 * meet their required degree with the edges that remain.
 */
bool backtrack(Graph* g,int index){
    /* Every selected fence has two endpoints, so the degree sum is even. */
    /* An odd sum is rejected here, before any edge is decided. */
    if(index == 0){
        int degree_sum = 0;
        for(int i = 0; i < g->rows * g->columns; i++){
            degree_sum += g->vertices[i].degree;
        }
        if(degree_sum % 2 != 0){
            return false;
        }
    }
    if(index == g->edge_num){
        for(int i = 0;i < g->rows* g->columns;i++){
            if(g->current_degree[FlattenIndex(g->vertices[i].x, g->vertices[i].y, g->columns)] != g->vertices[i].degree){
                return false;
            }
```

```
        } /* Leaf: every cell must meet its required degree exactly. */
        return true;
    }
    /* Branch 1: take edges[index] and test it in check_edge. */
    g->edges[index].is_selected = true;
    g->current_degree[FlattenIndex(g->edges[index].from.x, g->edges[index].from.y, g->columns)]++;
    g->current_degree[FlattenIndex(g->edges[index].to.x, g->edges[index].to.y, g->columns)]++;
    if(check_edge(g,index)){
        if(backtrack(g,index+1)){
            return true; /* A layout was found under this choice. */
        }
    }
    /* Branch 2: put the edge back and restore both endpoint degrees. */
    g->edges[index].is_selected = false;
    g->current_degree[FlattenIndex(g->edges[index].from.x, g->edges[index].from.y, g->columns)]--;
    g->current_degree[FlattenIndex(g->edges[index].to.x, g->edges[index].to.y, g->columns)]--;
    /* Skip this branch when either endpoint is already short of edges. */
    /* count_remaining_edges counts only candidates after this index. */
    Vertex from = g->edges[index].from;
    Vertex to = g->edges[index].to;
    if(g->current_degree[FlattenIndex(from.x, from.y, g->columns)]
          + count_remaining_edges(g, from, index) >= from.degree &&
       g->current_degree[FlattenIndex(to.x, to.y, g->columns)]
          + count_remaining_edges(g, to, index) >= to.degree){
        if(backtrack(g,index+1)){
            return true;
        }
    }
    return false; /* Neither branch produced a feasible layout. */
}



/*
 * Read n and m, build the candidate graph, and print one layout.
 * The drawing, or the line "No Solution", follows "The answer is:".
 */
int main(void){
    Graph g;
    int rows,columns;
    scanf("%d %d",&rows,&columns);
    init_Graph(&g,rows,columns);
    printf("The answer is:\n");
    if(backtrack(&g,0)){
        show_result(&g);
    }
    else{
        printf("No Solution\n");
    }
}
```

A.3 Windows Batch Timing Driver (`testbench_time.c`)

```
/*
 * File: testbench_time.c
 * Description: Windows driver. It runs recover_the_design.exe once per
 * .has file in TEST_DIR and writes ID, size, and seconds
 * to benchmark_results_100.csv. This file includes windows.h
 * and is compiled on Windows, not on Linux.
 */


#define _CRT_SECURE_NO_WARNINGS

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <windows.h> /* FindFirstFile, CreateProcess, QueryPerformanceCounter. */


#define PROGRAM_PATH "recover_the_design.exe" /* Solver launched for every instance. */
#define TEST_DIR "../data/Hashi_Puzzles/100" /* Directory of .has boards. */
#define RESULT_FILE "benchmark_results_100.csv" /* One row per board. */
```

```c

#define MAX_PATH_LEN 1024 /* Capacity of the path and command buffers. */
#define TIMEOUT_MS 60000 /* Kill a run that is still going after 60 seconds. */

/* Rows and columns read from the first line of one .has file. */
typedef struct {
    int rows; /* Board height. */
    int columns; /* Board width. */
} InstanceSize;


/*
 * Open one puzzle file and read its first two integers.
 * A missing file yields rows = 0 and columns = 0.
 */
InstanceSize get_instance_size(const char* filename) {
    InstanceSize size = {0, 0};

    FILE* fp = fopen(filename, "r");

    if (fp == NULL) {
        return size; /* Caller prints the size as Unknown. */
    }

    fscanf(fp, "%d %d", &size.rows, &size.columns); /* Header only; the grid is left for the solver. */

    fclose(fp);

    return size;
}


/*
 * Run the solver on one file and return the wall-clock seconds.
 * stdin is the puzzle file and stdout is discarded to temp_output.txt.
 * The return is negative when the process cannot be started, when the
 * wait fails, or when the run exceeds TIMEOUT_MS.
 */
double run_test(const char* filename) {
    char command[MAX_PATH_LEN * 3];
    char output_file[MAX_PATH_LEN];

    snprintf(output_file, sizeof(output_file),
            "temp_output.txt"); /* Solver stdout is thrown away after timing. */

    snprintf(command, sizeof(command),
            "cmd.exe /C \"\"%s\" < \"%s\" > \"%s\"\"",
            PROGRAM_PATH, filename, output_file);

    STARTUPINFOA startup;
    PROCESS_INFORMATION process;

    ZeroMemory(&startup, sizeof(startup)); /* CreateProcess requires a zeroed STARTUPINFO. */
    ZeroMemory(&process, sizeof(process));

    startup.cb = sizeof(startup);

    LARGE_INTEGER frequency, start, end;

    QueryPerformanceFrequency(&frequency); /* Ticks per second of the performance counter. */
    QueryPerformanceCounter(&start); /* Timestamp just before the child is created. */

    BOOL success = CreateProcessA(
        NULL, /* The executable is the first token of command. */
        command, /* cmd.exe redirects stdin from the puzzle file. */
        NULL,
        NULL,
        FALSE,
        CREATE_NO_WINDOW, /* No extra console for the child. */
        NULL,
        NULL,
        &startup,
        &process
    );

```

```
    if (!success) {
        printf("Failed to launch: %s\n", filename);
        return -1.0; /* CreateProcess failed; nothing to wait for. */
    }

    DWORD wait_result = WaitForSingleObject(
        process.hProcess,
        TIMEOUT_MS /* Give up once the limit has passed. */
    );

    if (wait_result == WAIT_TIMEOUT) {
        TerminateProcess(process.hProcess, 1); /* The board did not finish in time. */
        WaitForSingleObject(process.hProcess, INFINITE);

        CloseHandle(process.hProcess);
        CloseHandle(process.hThread);

        remove(output_file); /* The redirected stdout is not part of the result. */

        return -1.0; /* Counted as Timeout or Error in the CSV. */
    }

    QueryPerformanceCounter(&end); /* Timestamp after the child has exited. */

    double runtime =
        (double)(end.QuadPart - start.QuadPart)
        / frequency.QuadPart;

    CloseHandle(process.hProcess);
    CloseHandle(process.hThread);

    remove(output_file);

    if (wait_result != WAIT_OBJECT_0) {
        return -1.0; /* The wait ended for a reason other than exit or timeout. */
    }

    return runtime; /* Elapsed seconds, including process creation. */
}


/*
 * Walk every .has file in TEST_DIR, time the solver, and append a CSV row.
 * Directory entries are skipped. A negative runtime is written as
 * Timeout/Error instead of a number.
 */
int main(void) {
    WIN32_FIND_DATAA find_data;
    HANDLE handle;

    char search_path[MAX_PATH_LEN];

    snprintf(search_path, sizeof(search_path),
            "%s\\*.has", TEST_DIR); /* Non-recursive: only this directory. */

    handle = FindFirstFileA(search_path, &find_data);

    if (handle == INVALID_HANDLE_VALUE) {
        printf("No .has files found in %s\n", TEST_DIR); /* Nothing to time. */
        return 1;
    }

    /* Truncate any previous CSV so this run stands alone. */
    FILE* csv = fopen(RESULT_FILE, "w");

    if (csv == NULL) {
        printf("Cannot create CSV file.\n"); /* Leave the directory handle closed. */
        FindClose(handle); /* End the directory walk. */
        return 1;
    }

    fprintf(csv, "ID,Size,Runtime(s)\n"); /* Header consumed by the plotting script. */

    int id = 0; /* CSV identifier, starting at 1 for the first file. */

```

```c
    /* Files arrive in the order FindFirstFile / FindNextFile return them. */
    do {
        if (find_data.dwFileAttributes &
            FILE_ATTRIBUTE_DIRECTORY) {
            continue; /* FindFirstFile also returns the directory itself. */
        }

        /* Join the directory and the file name with a Windows separator. */
        char filename[MAX_PATH_LEN];

        snprintf(filename, sizeof(filename),
                "%s\\%s", TEST_DIR, find_data.cFileName);

        InstanceSize size = get_instance_size(filename); /* Used only for the size column. */

        double runtime = run_test(filename); /* Negative means timeout or launch failure. */

        id++; /* One identifier per file, including timeouts. */

        /* Printed both on the console and in the size column. */
        char size_string[64];

        if (size.rows > 0 && size.columns > 0) {
            snprintf(size_string, sizeof(size_string),
                    "%dx%d", size.rows, size.columns);
        }
        else {
            strcpy(size_string, "Unknown"); /* The header could not be read. */
        }

        printf(
            "ID: %d | File: %s | Size: %s | Time: ",
            id, find_data.cFileName, size_string
        );

        /* A negative runtime is a launch failure or a 60 second timeout. */
        if (runtime >= 0) {
            printf("%.6f s\n", runtime); /* Same clock reading that goes into the CSV. */

            fprintf(csv, "%d,%s,%.9f\n",
                   id, size_string, runtime);
        }
        else {
            printf("Timeout or Error\n"); /* The CSV stores the words, not a sentinel number. */

            fprintf(csv, "%d,%s,Timeout/Error\n",
                   id, size_string);
        }

    } while (FindNextFileA(handle, &find_data));

    FindClose(handle);
    fclose(csv); /* Flush every row before the process exits. */

    printf("\nAll tests completed.\n"); /* Every file in the directory has a row. */
    printf("Results saved to: %s\n", RESULT_FILE); /* Path of the CSV just closed. */

    return 0;
}
```

A.4 Pure Backtracking Profiler (`testbench_profile.c`)

```c
/*
 * File: testbench_profile.c
 * Description: Instrumented replay of the solver in recover_the_design.c.
 * Reads one instance from stdin, runs the same pruned search
 * while counting recursive calls, and prints one summary line:
 * rows cols |V| |E| |E*| calls time_us solved
 * Build: gcc -O2 -o testbench_profile testbench_profile.c
 * Ablation: add -DNO_LOOKAHEAD to drop the lower-bound test on the
 * exclusion branch; the search gives up after CALL_LIMIT calls.
 */

```

```c
#define _POSIX_C_SOURCE 199309L
#include <time.h>
#include "recover_the_design.h"

#define CALL_LIMIT 20000000LL /* Abort budget, reported as solved = -1 */

static long long call_count = 0; /* Number of search-tree nodes visited */

/*
 * Identical to backtrack() in recover_the_design.c except for the counter.
 */
bool backtrack_counted(Graph* g, int index) {
    call_count++;

    /* Root parity check (handshaking lemma) */
    if (index == 0) {
        int degree_sum = 0;
        for (int i = 0; i < g->rows * g->columns; i++) {
            degree_sum += g->vertices[i].degree;
        }
        if (degree_sum % 2 != 0) {
            return false;
        }
    }

    /* Leaf: every connector must meet its degree exactly */
    if (index == g->edge_num) {
        for (int i = 0; i < g->rows * g->columns; i++) {
            if (g->current_degree[i] != g->vertices[i].degree) {
                return false;
            }
        }
        return true;
    }

    int from_idx = FlattenIndex(
        g->edges[index].from.x, g->edges[index].from.y, g->columns);
    int to_idx = FlattenIndex(
        g->edges[index].to.x, g->edges[index].to.y, g->columns);

    /* Inclusion branch */
    g->edges[index].is_selected = true;
    g->current_degree[from_idx]++;
    g->current_degree[to_idx]++;
    if (check_edge(g, index) && call_count < CALL_LIMIT &&
        backtrack_counted(g, index + 1)) {
        return true;
    }

    /* Exclusion branch with lookahead lower bound */
    g->edges[index].is_selected = false;
    g->current_degree[from_idx]--;
    g->current_degree[to_idx]--;
    Vertex from = g->edges[index].from;
    Vertex to = g->edges[index].to;
#ifdef NO_LOOKAHEAD
    /* Ablation: always explore the exclusion branch */
    bool lookahead_ok = true;
#else
    bool lookahead_ok =
        g->current_degree[from_idx] +
            count_remaining_edges(g, from, index) >= from.degree &&
        g->current_degree[to_idx] +
            count_remaining_edges(g, to, index) >= to.degree;
#endif
    if (lookahead_ok && call_count < CALL_LIMIT &&
        backtrack_counted(g, index + 1)) {
        return true;
    }
    return false;
}

int main(void) {
    Graph g;
    int rows, columns;
```

```c
    if (scanf("%d %d", &rows, &columns) != 2) {
        return 1;
    }
    init_Graph(&g, rows, columns);

    /* Time only the search, not input parsing */
    struct timespec t0, t1;
    clock_gettime(CLOCK_MONOTONIC, &t0);
    bool ok = backtrack_counted(&g, 0);
    clock_gettime(CLOCK_MONOTONIC, &t1);
    double us = (t1.tv_sec - t0.tv_sec) * 1e6 + (t1.tv_nsec - t0.tv_nsec) / 1e3;

    /* Count fences in the returned drawing */
    int selected = 0;
    for (int i = 0; i < g.edge_num; i++) {
        selected += g.edges[i].is_selected;
    }
    int status = ok ? 1 : (call_count >= CALL_LIMIT ? -1 : 0);
    printf("%d %d %d %d %d %lld %.3f %d\n", rows, columns, g.vertex_num,
           g.edge_num, ok ? selected : 0, call_count, us, status);
    return 0;
}
```

A.5 Structural Output Verifier (`verify_output.py`)

```python
# File: verify_output.py
# Description: Independent checker for one solver output.
# Usage: python3 verify_output.py <input file> <output file>
# Checks the output format and row-major order, that every printed side
# points at the nearest connector and is matched from the other end,
# that every degree is met, and that no two fences cross in an empty cell.
# "No Solution" is accepted only when the degree sum is odd, or when
# exhaustive search is affordable (<= 22 candidate edges) and finds nothing.

import itertools
import sys


def read_grid(path):
    nums = list(map(int, open(path).read().split()))
    n, m = nums[0], nums[1]
    return n, m, [nums[2 + i * m: 2 + (i + 1) * m] for i in range(n)]


def nearest(g, n, m, i, j, di, dj):
    # First connector strictly beyond (i, j) in direction (di, dj), or None.
    i, j = i + di, j + dj
    while 0 <= i < n and 0 <= j < m:
        if g[i][j] > 0:
            return (i, j)
        i, j = i + di, j + dj
    return None


def segments_cross(h, v):
    # h = (row, c1, c2), v = (col, r1, r2); open-interior intersection.
    return h[1] < v[0] < h[2] and v[1] < h[0] < v[2]


def check_drawing(n, m, g, lines):
    conns = [(i, j) for i in range(n) for j in range(m) if g[i][j] > 0]
    if len(lines) != len(conns):
        return "wrong number of lines"
    status = {}
    for (i, j), line in zip(conns, lines):
        if line != line.strip() or "  " in line:
            return "bad spacing"
        r, c, up, down, left, right = map(int, line.split())
        if (r - 1, c - 1) != (i, j):
            return "not row-major"
        status[(i, j)] = (up, down, left, right)
    hs, vs = [], []
    dirs = [(-1, 0), (1, 0), (0, -1), (0, 1)]
```

```
    for (i, j), s in status.items():
        if sum(s) != g[i][j]:
            return f"degree mismatch at {(i + 1, j + 1)}"
        for k, (di, dj) in enumerate(dirs):
            if not s[k]:
                continue
            t = nearest(g, n, m, i, j, di, dj)
            if t is None or not status[t][k ^ 1]:
                return f"dangling fence at {(i + 1, j + 1)}"
            if k == 3:
                hs.append((i, j, t[1]))
            if k == 1:
                vs.append((j, i, t[0]))
    for h in hs:
        for v in vs:
            if segments_cross(h, v):
                return "fences cross"
    return None


def has_solution(n, m, g):
    # Exhaustive search over candidate edges; only used for tiny inputs.
    edges = []
    for i in range(n):
        for j in range(m):
            if g[i][j] > 0:
                for di, dj in [(0, 1), (1, 0)]:
                    t = nearest(g, n, m, i, j, di, dj)
                    if t:
                        edges.append(((i, j), t))
    if len(edges) > 22:
        return None
    for mask in range(1 << len(edges)):
        sel = [edges[k] for k in range(len(edges)) if mask >> k & 1]
        deg = {}
        for a, b in sel:
            deg[a] = deg.get(a, 0) + 1
            deg[b] = deg.get(b, 0) + 1
        if any(deg.get((i, j), 0) != g[i][j] for i in range(n) for j in range(m)):
            continue
        hs = [(a[0], a[1], b[1]) for a, b in sel if a[0] == b[0]]
        vs = [(a[1], a[0], b[0]) for a, b in sel if a[1] == b[1]]
        if not any(segments_cross(h, v) for h, v in itertools.product(hs, vs)):
            return True
    return False


def main():
    n, m, g = read_grid(sys.argv[1])
    lines = [
        l for l in open(sys.argv[2]).read().splitlines()
        if l.strip() != "The answer is:"
    ]
    if lines == ["No Solution"]:
        if sum(map(sum, g)) % 2 == 1:
            print("OK (odd degree sum)")
        elif has_solution(n, m, g) is False:
            print("OK (exhaustive check)")
        else:
            print("UNVERIFIED No Solution")
            sys.exit(1)
        return
    err = check_drawing(n, m, g, lines)
    print("OK" if err is None else "FAIL: " + err)
    sys.exit(err is not None)


if __name__ == "__main__":
    main()
```

A.6 Benchmark and Layout Visualization Script (`visualization.py`)

```
# File: visualization.py
```

```python
# Description: Generate benchmark and layout figures for Project 3 (Recover the Design):
# 1) End-to-end runtime across the 360 Hashi_100 instances (benchmark_results_100.csv)
# 2) Scaling of candidate edges, recursive calls, and pure solver time across L05-L20
# 3) Showcase of recovered non-crossing fence layouts on L08, L12, and Instance #1

import csv
import os
import re
import subprocess
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

plt.rcParams["font.sans-serif"] = ["DejaVu Sans"]
plt.rcParams["axes.unicode_minus"] = False

PRIMARY = "#1C375A"
ACCENT = "#B08034"
PROG = "#346958"
MUTED = "#5F6976"
RED = "#A83232"

os.makedirs("visualizations", exist_ok=True)

# ------------------------------------------------------------
# 1. Figure: Runtime trajectory + histogram across 360 instances
# ------------------------------------------------------------
bench_rows = []
with open("benchmark_results_100.csv", newline="", encoding="utf-8") as f:
    for r in csv.DictReader(f):
        bench_rows.append((int(r["ID"]), r["Size"], float(r["Runtime(s)"]) * 1000.0))

ids = [r[0] for r in bench_rows]
runtimes_ms = [r[2] for r in bench_rows]

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11.5, 4.4), gridspec_kw={"width_ratios": [1.55, 1.0]})

ax1.scatter(ids, runtimes_ms, s=14, color=PRIMARY, alpha=0.68, label="Instance runtime (ms)")
window = 15
rolling = np.convolve(runtimes_ms, np.ones(window) / window, mode="valid")
ax1.plot(range(window, len(runtimes_ms) + 1), rolling, color=ACCENT, linewidth=2.0, label="15-instance moving average")
ax1.axhline(np.mean(runtimes_ms), color=PROG, linestyle="--", linewidth=1.3, label=f"Mean = {np.mean(runtimes_ms):.2f} ms")

ax1.set_xlabel("Benchmark Instance ID (1 to 360, 16x16 Grid, 100 Connectors)")
ax1.set_ylabel("Execution Time (ms)")
ax1.set_title("(a) Per-Instance Execution Time Across 360 Benchmark Cases")
ax1.set_xlim(0, 365)
ax1.set_ylim(35, 198)
ax1.legend(loc="upper left", fontsize=8.5, frameon=True)
ax1.grid(True, linestyle="--", alpha=0.4)

ax2.hist(runtimes_ms, bins=24, color=PRIMARY, edgecolor="white", alpha=0.85)
ax2.axvline(np.median(runtimes_ms), color=ACCENT, linewidth=2.0, linestyle="-", label=f"Median = {np.median(runtimes_ms):.2f} ms")
ax2.axvline(np.mean(runtimes_ms), color=PROG, linewidth=2.0, linestyle="--", label=f"Mean = {np.mean(runtimes_ms):.2f} ms")
ax2.axvline(np.percentile(runtimes_ms, 95), color=RED, linewidth=1.6, linestyle=":", label=f"95th pct = {np.percentile(runtimes_ms, 95):.2f} ms")
ax2.set_xlabel("Execution Time (ms)")
ax2.set_ylabel("Number of Instances")
ax2.set_title("(b) Empirical Runtime Distribution (N = 360)")
ax2.legend(loc="upper right", fontsize=8.5, frameon=True)
ax2.grid(True, linestyle="--", alpha=0.4)

plt.tight_layout()
plt.savefig("visualizations/hashi100_runtime_overview.pdf")
plt.savefig("visualizations/hashi100_runtime_overview.png", dpi=300)
plt.close()

# ------------------------------------------------------------
# 2. Figure: Scaling on L05-L20 (5x5 to 20x20 grids)
# ------------------------------------------------------------
with open("../data/Hashi_Puzzles/recover_the_design_large_tests.txt", encoding="utf-8") as f:
    text = f.read().replace("\r\n", "\n")
pattern = r"\u6837\u4f8b (L\d+)\uff1a([^\n]+)\n(?:[^\n]+\n)?\u8f93\u5165\uff1a\n((?:\d+[^\n]*\n)+)"
```

```
cases = re.findall(pattern, text)

def profile(inp, reps=7):
    # Run testbench_profile multiple times and keep the minimum time.
    runs = [subprocess.run(["./testbench_profile"], input=inp, text=True,
                           capture_output=True).stdout.split() for _ in range(reps)]
    return min(runs, key=lambda o: float(o[6]))

dims, verts, cand_edges, sel_edges, rec_calls, solve_us = [], [], [], [], [], []
for name, desc, inp in cases:
    if not name.startswith("L"):
        continue
    r, c, v, e, s, calls, us, ok = profile(inp)
    dims.append(int(r))
    verts.append(int(v))
    cand_edges.append(int(e))
    sel_edges.append(int(s))
    rec_calls.append(int(calls))
    solve_us.append(float(us) / 1000.0)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11.5, 4.3))

ax1.plot(dims, verts, marker="o", color=PRIMARY, linewidth=2.0, label=r"Connectors $|V|$")
ax1.plot(dims, cand_edges, marker="s", color=ACCENT, linewidth=2.0, label=r"Candidate edges $|E|$")
ax1.plot(dims, sel_edges, marker="^", color=PROG, linewidth=2.0, label=r"Selected fences $|E^*|$")
ax1.set_xlabel(r"Grid Dimension $n \times n$ ($n \in [5, 20]$)")
ax1.set_ylabel("Count")
ax1.set_title("(a) Structural Growth of Graph Vertices and Edges (L05-L20)")
ax1.set_xticks(dims)
ax1.legend(frameon=True, fontsize=9)
ax1.grid(True, linestyle="--", alpha=0.4)

ax2_twin = ax2.twinx()
l1 = ax2.plot(dims, rec_calls, marker="o", color=PRIMARY, linewidth=2.0, label="Recursive calls (log scale)")
l2 = ax2_twin.plot(dims, solve_us, marker="D", color=ACCENT, linewidth=2.0, linestyle="--", label="Pure search time (ms, log
      scale)")
ax2.set_yscale("log")
ax2_twin.set_yscale("log")
ax2.set_xlabel(r"Grid Dimension $n \times n$ ($n \in [5, 20]$)")
ax2.set_ylabel("Backtrack Recursive Calls (log scale)", color=PRIMARY)
ax2_twin.set_ylabel("Pure Backtracking Time (ms, log scale)", color=ACCENT)
ax2.set_title("(b) Search Tree Nodes and Runtime Scaling (L05-L20)")
ax2.set_xticks(dims)
ax2.grid(True, linestyle="--", alpha=0.4)

lines = l1 + l2
labels = [l.get_label() for l in lines]
ax2.legend(lines, labels, loc="upper left", frameon=True, fontsize=8.5)

plt.tight_layout()
plt.savefig("visualizations/large_tests_scaling.pdf")
plt.savefig("visualizations/large_tests_scaling.png", dpi=300)
plt.close()

# ------------------------------------------------------------
# 3. Figure: Showcase of recovered fence layouts
# ------------------------------------------------------------
subprocess.run(["gcc", "-O2", "-o", "/tmp/rtd_vis", "recover_the_design.c"], check=True)

def parse_grid_text(txt):
    nums = list(map(int, txt.split()))
    n, m = nums[0], nums[1]
    grid = [nums[2 + i * m : 2 + (i + 1) * m] for i in range(n)]
    return n, m, grid

def solve_and_draw(ax, txt, title_str, node_size, font_size, lw_fence):
    n, m, grid = parse_grid_text(txt)
    proc = subprocess.run(["/tmp/rtd_vis"], input=txt, text=True, capture_output=True, check=True)
    status = {}
    for line in proc.stdout.splitlines():
        parts = line.strip().split()
        if len(parts) == 6 and all(p.lstrip("-").isdigit() for p in parts):
            r, c, up, down, left, right = map(int, parts)
            status[(r - 1, c - 1)] = (up, down, left, right)

```

```
    # Background grid lines
    for i in range(n):
        ax.plot([0, m - 1], [i, i], color="#E5E9F0", linewidth=0.6, zorder=1)
    for j in range(m):
        ax.plot([j, j], [0, n - 1], color="#E5E9F0", linewidth=0.6, zorder=1)

    # Candidate edges (dashed) and selected fences (solid)
    sel_count = 0
    cand_count = 0
    for i in range(n):
        for j in range(m):
            if grid[i][j] > 0:
                nj = j + 1
                while nj < m and grid[i][nj] == 0:
                    nj += 1
                if nj < m:
                    cand_count += 1
                    if status.get((i, j), (0, 0, 0, 0))[3] == 1:
                        ax.plot([j, nj], [i, i], color=ACCENT, linewidth=lw_fence, solid_capstyle="round", zorder=3)
                        sel_count += 1
                    else:
                        ax.plot([j, nj], [i, i], color="#B8C2CC", linewidth=0.85, linestyle="--", zorder=2)
                ni = i + 1
                while ni < n and grid[ni][j] == 0:
                    ni += 1
                if ni < n:
                    cand_count += 1
                    if status.get((i, j), (0, 0, 0, 0))[1] == 1:
                        ax.plot([j, j], [i, ni], color=ACCENT, linewidth=lw_fence, solid_capstyle="round", zorder=3)
                        sel_count += 1
                    else:
                        ax.plot([j, j], [i, ni], color="#B8C2CC", linewidth=0.85, linestyle="--", zorder=2)

    xs, ys, degs = [], [], []
    for i in range(n):
        for j in range(m):
            if grid[i][j] > 0:
                xs.append(j)
                ys.append(i)
                degs.append(grid[i][j])
    ax.scatter(xs, ys, s=node_size, facecolor="#F4F6FA", edgecolor=PRIMARY, linewidth=1.35, zorder=4)
    for x, y, d in zip(xs, ys, degs):
        ax.text(x, y, str(d), ha="center", va="center", fontsize=font_size, fontweight="bold", color=PRIMARY, zorder=5)

    ax.set_xlim(-0.7, m - 0.3)
    ax.set_ylim(n - 0.3, -0.7)
    ax.set_aspect("equal")
    ax.set_xticks(range(0, m, 2 if m > 10 else 1))
    ax.set_xticklabels([str(x + 1) for x in range(0, m, 2 if m > 10 else 1)], fontsize=7.5)
    ax.set_yticks(range(0, n, 2 if n > 10 else 1))
    ax.set_yticklabels([str(y + 1) for y in range(0, n, 2 if n > 10 else 1)], fontsize=7.5)
    ax.set_title(f"{title_str}\n(|V|={len(xs)}, |E|={cand_count}, |E*|={sel_count})", fontsize=9.2, fontweight="bold", color
        =PRIMARY)

case_dict = {name: inp for name, desc, inp in cases}
with open("../data/Hashi_Puzzles/100_revised/Hs_16_100_25_00_001.has", encoding="utf-8") as f:
    hashi_001_txt = f.read()

fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(11.8, 4.1))
solve_and_draw(ax1, case_dict["L08"], "(a) Constructed Board L08 (8x8)", node_size=145, font_size=7.5, lw_fence=2.3)
solve_and_draw(ax2, case_dict["L12"], "(b) Constructed Board L12 (12x12)", node_size=85, font_size=6.5, lw_fence=1.9)
solve_and_draw(ax3, hashi_001_txt, "(c) Benchmark Instance #1 (16x16)", node_size=55, font_size=5.8, lw_fence=1.7)

plt.tight_layout()
plt.savefig("visualizations/recovered_layouts_showcase.pdf")
plt.savefig("visualizations/recovered_layouts_showcase.png", dpi=300)
plt.close()
print("All figures generated successfully.")
```